



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	14 / Tipe Data Set

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Pengenalan dan Mendefinisikan Set

Set adalah tipe data Python untuk menyimpan kumpulan data unik (tidak ada duplikat).

Sifat-sifat Set:

- Isi Set disebut anggota (member).
- Anggota Set harus bersifat immutable, seperti integer, float, string, dan tuple.
- List dan dictionary tidak bisa dimasukkan ke Set karena bersifat mutable.
- Set sendiri bersifat mutable, sehingga isi Set dapat ditambah atau dikurangi.
- Set dapat dibuat menggunakan notasi {} atau fungsi set().

```
1  # dengan menggunakan {}
2  bilangan_genap = {2, 4, 6, 8, 10, 12}
3  bilangan_ganjil = {1, 3, 5, 7, 9, 11}
4
5  # dengan menggunakan fungsi set()
6  pernah_ke_bulan = set('Neil Armstrong', 'Buzz Aldrin')
```

Gambar 1.1: Penerapan Set pada Python Menggunakan Kurung Kurawal {} dan Fungsi set()

Untuk mendefinisikan Set kosong tidak dapat menggunakan notasi {}, tapi harus menggunakan fungsi set(), berikut contoh nya:

```
1  # dengan fungsi set()
2  pernah_ke_mars = set() # menghasilkan set kosong
3  data = {} # ini akan menghasilkan dictionary kosong
```

Gambar 1.2: Penerapan Set Kosong dengan Fungsi set() pada Python

Pengaksesan Set

Set tidak memiliki indeks, karenanya tidak dapat mengakses anggota-anggota dari sebuah set secara langsung menggunakan indeks.

```
1  nim={'71200120', '71200195', '71200214'}
2  jumlah_nim= len(nim)
3  print(jumlah_nim) # akanmenghasilkan output 3
4
5  # tampilkan isi setsatu-persatu
6  for n in nim:
7      print(n)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
3
71200214
71200195
71200120
```

Gambar 1.3: Penggunaan Fungsi len() dan Perulangan for pada Set di Python

Urutan output pada Set dapat berbeda dari deklarasinya karena Set tidak memiliki indeks atau urutan tetap. Pada Set, posisi anggota tidak penting. Set juga bersifat mutable, sehingga anggotanya dapat ditambah atau dikurangi. Program berikut menunjukkan cara menambahkan anggota ke dalam Set menggunakan fungsi add().

```
1  # definisikan sebuah set kosong
2  plat_nomor = set()
3
4  # tambahkan plat nomor 'AB 1890 XA'
5  plat_nomor.add('AB 1890 XA')
6
7  # tambahkan plat nomor 'AD 6810 MT'
8  plat_nomor.add('AD 6810 MT')
9
10 # jumlah anggota di dalam Set
11 print(len(plat_nomor))
12
13 # tambahkan plat yang sama sekali lagi
14 plat_nomor.add('AB 1890 XA')
15
16 # tampilkan semua plat nomor
17 for plat in plat_nomor:
18     print(plat)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
2
AB 1890 XA
AD 6810 MT
PS D:\New folder - Copy\Coding> 
```

Gambar 1.4: Penambahan Anggota pada Set Menggunakan Fungsi add()

Set secara otomatis memeriksa duplikasi saat fungsi `add()` digunakan. Jika data sudah ada, maka anggota baru tidak akan ditambahkan. Untuk menghapus anggota Set dapat digunakan fungsi `discard()`, `remove()`, `pop()`, dan `clear()`. Program berikut menunjukkan contoh penghapusan anggota pada Set.

```

setpy > ...
1  bilangan_prima = {13, 23, 7, 29, 11, 5}
2
3  # hapus 5 dari set tersebut
4  bilangan_prima.remove(5)
5  print(bilangan_prima)
6
7  # hapus 97 (tidak ada)
8  bilangan_prima.discard(97)
9  print(bilangan_prima)
10
11 # ambil dan hapus salah satu
12 bilangan = bilangan_prima.pop()
13 print(bilangan)
14 print(bilangan_prima)
15
16 # kosongkan set
17 bilangan_prima.clear()
18 print(bilangan_prima)
  
```

```

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{29, 23, 7, 11, 13}
{29, 23, 7, 11, 13}
29
{23, 7, 11, 13}
set()
PS D:\New folder - Copy\Coding>
PS D:\New folder - Copy\Coding>
  
```

Gambar 1.5: Penghapusan Anggota Set Menggunakan Fungsi `remove()`

<code>discard()</code>	<code>remove()</code>	<code>pop()</code>	<code>clear()</code>
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Gambar 1.6: Fungsi-fungsi untuk menghapus anggota dari Set

Fungsi `discard()` tidak akan menghasilkan error jika anggota yang ingin dihapus tidak ada didalam set. Jika fungsi `pop()` akan mengambil salah satu anggota (secara acak) dan mengeluarkannya dari Set. Fungsi `pop()` berguna jika ingin memproses isi dari set satu-persatu tanpa memperdulikan urutan/posisi dari setiap anggota di dalam set.

Jika ingin mengubah nilai salah satu anggota di dalam set? Pada Set tidak bisa mengubah langsung nilai dari salah satu anggota didalam Set, yang bisa dilakukan adalah melakukan operasi penggantian (`replace`) dengan cara menghapus anggota yang mau diubah, Kemudian memasukan anggota baru yang nilainya sesuai dengan yang diinginkan. Berikut merupakan contoh penerapannya.

```
1 # Buat Set dari List
2 ikan = set(['koi', 'koki', 'kembung', 'salmon'])
3 print(ikan)
4 # ganti koi menjadi teri
5 ikan.remove('koi')
6 ikan.add('teri')
7 print(ikan)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{'koi', 'kembung', 'salmon', 'koi'}
{'salmon', 'teri', 'kembung', 'koi'}
PS D:\New folder - Copy\Coding> █
```

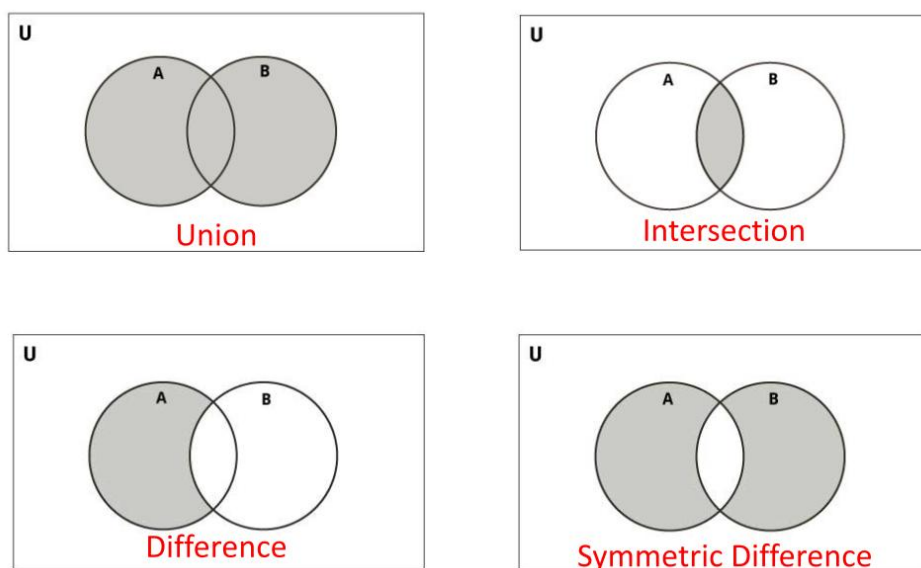
Gambar 1.7: Program Python untuk mengubah elemen pada set

Untuk mengubah nilai koi menjadi teri, data koi harus dihapus terlebih dahulu lalu ditambahkan data teri. Pada set, urutan data tidak tetap sehingga setelah penambahan atau penghapusan, urutan isi set bisa berubah.

Operasi-Operasi pada Set

Operasi-operasi set adalah operasi-operasi himpunan. Berikut daftar operasi set pada python:

- Union: Menggabungkan dua set menjadi satu menggunakan | atau union().
- Intersection: Menghasilkan irisan dua set menggunakan & atau intersection().
- Difference: Menghasilkan selisih dua set menggunakan - atau difference().
- Symmetric Difference: Menghasilkan gabungan dua set tanpa irisan menggunakan ^ atau symmetric_difference().



Gambar 1.8: Operasi-operasi pada Set

Operator Union

Berikut merupakan contoh penggunaan operator union dalam program python:

```
1  merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
2  merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}
3  # union dari merek_hp dan merek_ac
4  gabungan = merek_hp | merek_ac
5  # bisa juga menggunakan gabungan = merek_hp.union(merek_ac)
6  print(gabungan)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{'Apple', 'Panasonic', 'Samsung', 'LG', 'Daikin', 'Sony', 'Xiaomi'}
PS D:\New folder - Copy\Coding>
```

Gambar 1.9: Program Python operasi union pada set

Program menghasilkan set baru bernama gabungan dari penggabungan merek_hp dan merek_ac. Data yang sama seperti Samsung dan Sony hanya muncul satu kali karena anggota set harus unik.

Operator Intersection

Berikut merupakan contoh penggunaan operator Intersection dalam program python:

```
1  renang = {'siti', 'mail', 'ikhsan', 'upin', 'ipin'}
2  tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}
3  # suka renang dan tenis
4  renang_tenis = renang & tenis
5  print(reenang_tenis)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{'mail', 'ipin', 'upin'}
PS D:\New folder - Copy\Coding>
```

Gambar 2.0: Program Python operasi Intersection pada set

Program menghasilkan operasi intersection dari set renang dan tenis, yaitu anggota yang ada di kedua set sekaligus yaitu mail, upin, dan ipin.

Operator Difference

Berikut merupakan contoh penggunaan operator difference dalam program python:

```
1 # bisa berbahasa english
2 english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
3 # bisa berbahasa korea
4 korean = {'chaewon', 'yeona', 'erika', 'miko'}
5 # siapa yang hanya bisa bahasa korea?
6 only_korean = korean - english
7 print(only_korean)
8 # siapa yang hanya bisa bahasa english?
9 only_english = english - korean
10 print(only_english)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{'yeona', 'erika'}
{'desi', 'tono', 'takashi', 'evan'}
PS D:\New folder - Copy\Coding>
```

Gambar 2.1: Program Python operasi difference pada set

Operator difference menghasilkan set berisi anggota yang merupakan selisih dari dua set. Contohnya, mencari anggota yang hanya bisa bahasa Korea dari selisih set korean dan english, atau sebaliknya untuk bahasa Inggris.

Operator Symmetric Difference

Berikut merupakan contoh penggunaan operator Symmetric difference dalam program python:

```
1 english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
2 korean = {'chaewon', 'yeona', 'erika', 'miko'}
3 # hanya bisa bicara satu bahasa saja
4 one_language = english ^ korean
5 print(one_language)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\set.py"
{'yeona', 'evan', 'erika', 'takashi', 'tono', 'desi'}
PS D:\New folder - Copy\Coding>
```

Gambar 2.2: Program Python operasi symmetric difference pada set

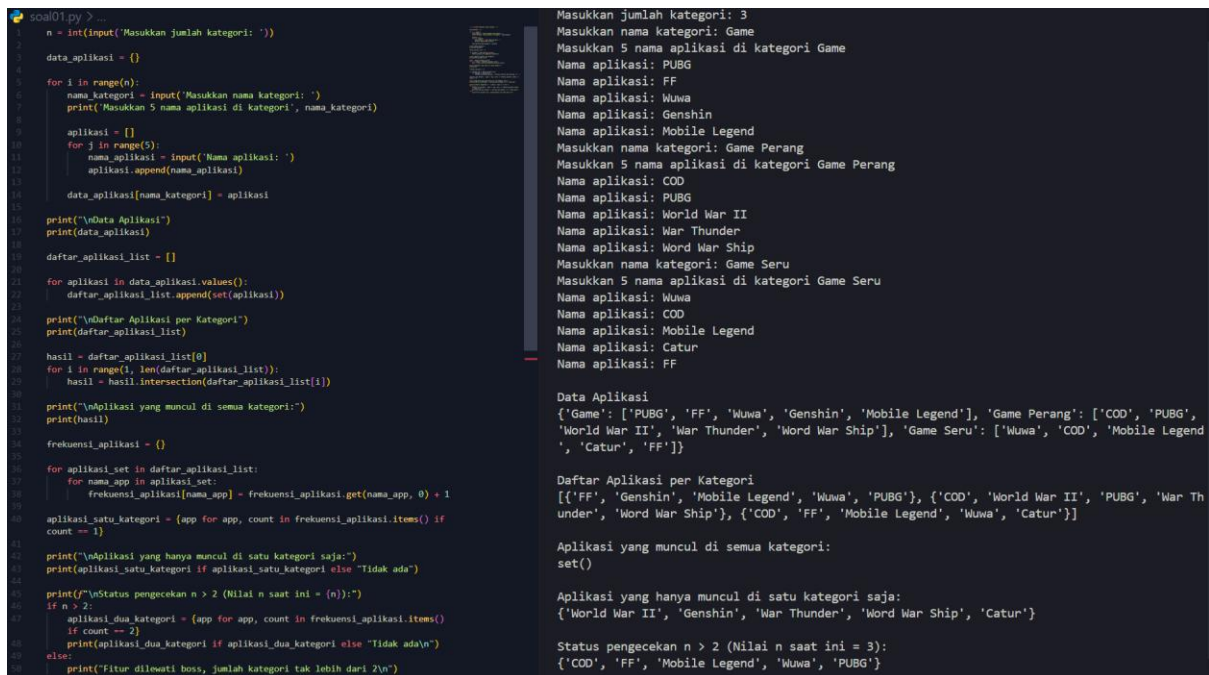
Operator symmetric difference menghasilkan set gabungan dari dua set tanpa anggota yang sama (irisan). Contohnya digunakan untuk mencari anggota yang hanya bisa satu bahasa saja. Secara umum dapat juga dihasilkan dari operasi berikut:

```
one_language = english.union(korean) - english.intersection(korean)
```

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01



```
soal01.py >
n = int(input('Masukkan jumlah kategori: '))

data_aplikasi = {}

for i in range(n):
    nama_kategori = input('Masukkan nama kategori: ')
    print('Masukkan 5 nama aplikasi di kategori', nama_kategori)

    aplikasi = []
    for j in range(5):
        nama_aplikasi = input('Nama aplikasi: ')
        aplikasi.append(nama_aplikasi)

    data_aplikasi[nama_kategori] = aplikasi

print('\nData Aplikasi')
print(data_aplikasi)

daftar_aplikasi_list = []

for aplikasi in data_aplikasi.values():
    daftar_aplikasi_list.append(set(aplikasi))

print('\nDaftar Aplikasi per Kategori')
print(daftar_aplikasi_list)

hasil = daftar_aplikasi_list[0]
for i in range(1, len(daftar_aplikasi_list)):
    hasil = hasil.intersection(daftar_aplikasi_list[i])

print('\nAplikasi yang muncul di semua kategori:')
print(hasil)

frekuensi_aplikasi = {}

for aplikasi_set in daftar_aplikasi_list:
    for nama_app in aplikasi_set:
        frekuensi_aplikasi[nama_app] = frekuensi_aplikasi.get(nama_app, 0) + 1

aplikasi_satu_kategori = {app for app, count in frekuensi_aplikasi.items() if count == 1}

print('\nAplikasi yang hanya muncul di satu kategori saja:')
print(aplikasi_satu_kategori if aplikasi_satu_kategori else 'Tidak ada')

print(f'\nStatus pengecekan n > 2 (Nilai n saat ini = {n}):')
if n > 2:
    aplikasi_dua_kategori = {app for app, count in frekuensi_aplikasi.items() if count == 2}
    print(aplikasi_dua_kategori if aplikasi_dua_kategori else 'Tidak ada')
else:
    print('Fitur dilewati boss, jumlah kategori tak lebih dari 2!')

Masukkan jumlah kategori: 3
Masukkan nama kategori: Game
Masukkan 5 nama aplikasi di kategori Game
Nama aplikasi: PUBG
Nama aplikasi: FF
Nama aplikasi: Wuwa
Nama aplikasi: Genshin
Nama aplikasi: Mobile Legend
Masukkan nama kategori: Game Perang
Masukkan 5 nama aplikasi di kategori Game Perang
Nama aplikasi: COD
Nama aplikasi: PUBG
Nama aplikasi: World War II
Nama aplikasi: War Thunder
Nama aplikasi: Word War Ship
Masukkan nama kategori: Game Seru
Masukkan 5 nama aplikasi di kategori Game Seru
Nama aplikasi: Wuwa
Nama aplikasi: COD
Nama aplikasi: Mobile Legend
Nama aplikasi: Catur
Nama aplikasi: FF

Data Aplikasi
{'Game': ['PUBG', 'FF', 'Wuwa', 'Genshin', 'Mobile Legend'], 'Game Perang': ['COD', 'PUBG', 'World War II', 'War Thunder', 'Word War Ship'], 'Game Seru': ['Wuwa', 'COD', 'Mobile Legend', 'Catur', 'FF']}

Daftar Aplikasi per Kategori
[{'FF', 'Genshin', 'Mobile Legend', 'Wuwa', 'PUBG'}, {'COD', 'World War II', 'PUBG', 'War Thunder', 'Word War Ship'}, {'COD', 'FF', 'Mobile Legend', 'Wuwa', 'Catur'}]

Aplikasi yang muncul di semua kategori:
set()

Aplikasi yang hanya muncul di satu kategori saja:
{'World War II', 'Genshin', 'War Thunder', 'Word War Ship', 'Catur'}

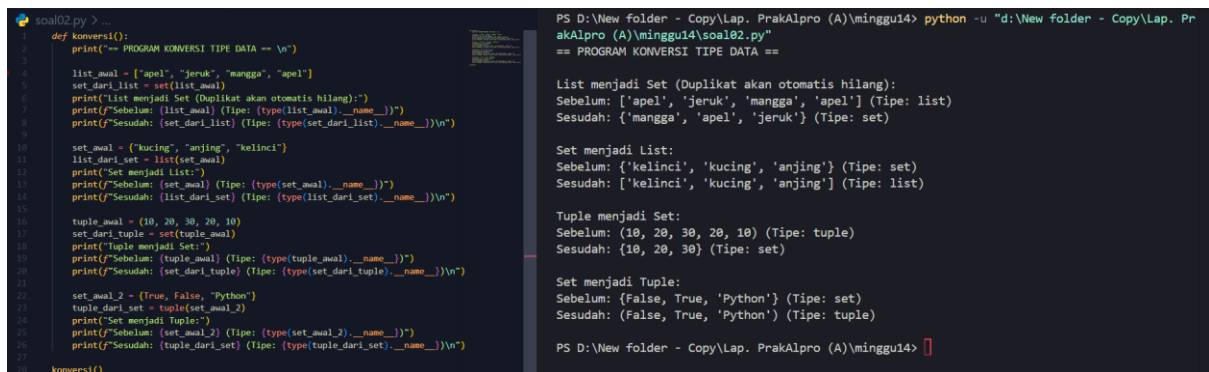
Status pengecekan n > 2 (Nilai n saat ini = 3):
{'COD', 'FF', 'Mobile Legend', 'Wuwa', 'PUBG'}
```

Gambar 2.3: Source Code soal 01

Penjelasan:

Ini adalah program dari kasus Play Store di materi dengan tambahan beberapa fitur. Program dimulai dengan meminta pengguna memasukkan jumlah kategori aplikasi lalu setiap kategori diisi 5 nama aplikasi yang disimpan ke dalam dictionary, misalnya {'Game': ['PUBG', 'FF', ...]}. Setelah itu, setiap daftar aplikasi diubah menjadi set agar duplikat otomatis dihapus sehingga terbentuk list berisi kumpulan aplikasi unik pada tiap kategori. Program kemudian menggunakan **intersection()** untuk mencari aplikasi yang muncul di semua kategori, dan dari hasil output diperoleh **set()** yang berarti tidak ada aplikasi yang sama di seluruh kategori. Selanjutnya dibuat dictionary frekuensi untuk menghitung berapa kali setiap aplikasi muncul pada kategori yang berbeda. Berdasarkan frekuensi tersebut, program menampilkan aplikasi yang hanya muncul di satu kategori, seperti "World War II", "Genshin", "War Thunder", "Word War Ship", dan "Catur". Terakhir, jika jumlah kategori lebih dari 2, program akan mencari aplikasi yang muncul tepat pada dua kategori, dengan hasil seperti "COD", "FF", "Mobile Legend", "Wuwa", dan "PUBG"; namun jika kategori tidak lebih dari 2 maka fitur tersebut dilewati.

SOAL 02



```
soal02.py >
def konversi():
    print("-- PROGRAM KONVERSI TIPE DATA -- \n")

    list_awal = ["apel", "jeruk", "mangga", "apel"]
    set_dari_list = set(list_awal)
    print("List menjadi Set (Duplikat akan otomatis hilang):")
    print(f"Sebelum: {list_awal} (Tipe: {type(list_awal).__name__})")
    print(f"Sesudah: {set_dari_list} (Tipe: {type(set_dari_list).__name__})\n")

    set_awal = {"kucing", "anjing", "kelinci"}
    list_dari_set = list(set_awal)
    print("Set menjadi List:")
    print(f"Sebelum: {set_awal} (Tipe: {type(set_awal).__name__})")
    print(f"Sesudah: {list_dari_set} (Tipe: {type(list_dari_set).__name__})\n")

    tuple_awal = (10, 20, 30, 20, 10)
    set_dari_tuple = set(tuple_awal)
    print("Tuple menjadi Set:")
    print(f"Sebelum: {tuple_awal} (Tipe: {type(tuple_awal).__name__})")
    print(f"Sesudah: {set_dari_tuple} (Tipe: {type(set_dari_tuple).__name__})\n")

    set_awal_2 = {True, False, "Python"}
    tuple_dari_set = tuple(set_awal_2)
    print("Set menjadi Tuple:")
    print(f"Sebelum: {set_awal_2} (Tipe: {type(set_awal_2).__name__})")
    print(f"Sesudah: {tuple_dari_set} (Tipe: {type(tuple_dari_set).__name__})\n")

konversi()
```

```
PS D:\New folder - Copy\Lap. PrakaIpro (A)\minggui4> python -u "d:\New folder - Copy\Lap. PrakaIpro (A)\minggui4\soal02.py"
== PROGRAM KONVERSI TIPE DATA ==

List menjadi Set (Duplikat akan otomatis hilang):
Sebelum: ['apel', 'jeruk', 'mangga', 'apel'] (Tipe: list)
Sesudah: {'mangga', 'apel', 'jeruk'} (Tipe: set)

Set menjadi List:
Sebelum: {'kelinci', 'kucing', 'anjing'} (Tipe: set)
Sesudah: ['kelinci', 'kucing', 'anjing'] (Tipe: list)

Tuple menjadi Set:
Sebelum: (10, 20, 30, 20, 10) (Tipe: tuple)
Sesudah: {10, 20, 30} (Tipe: set)

Set menjadi Tuple:
Sebelum: {False, True, 'Python'} (Tipe: set)
Sesudah: (False, True, 'Python') (Tipe: tuple)

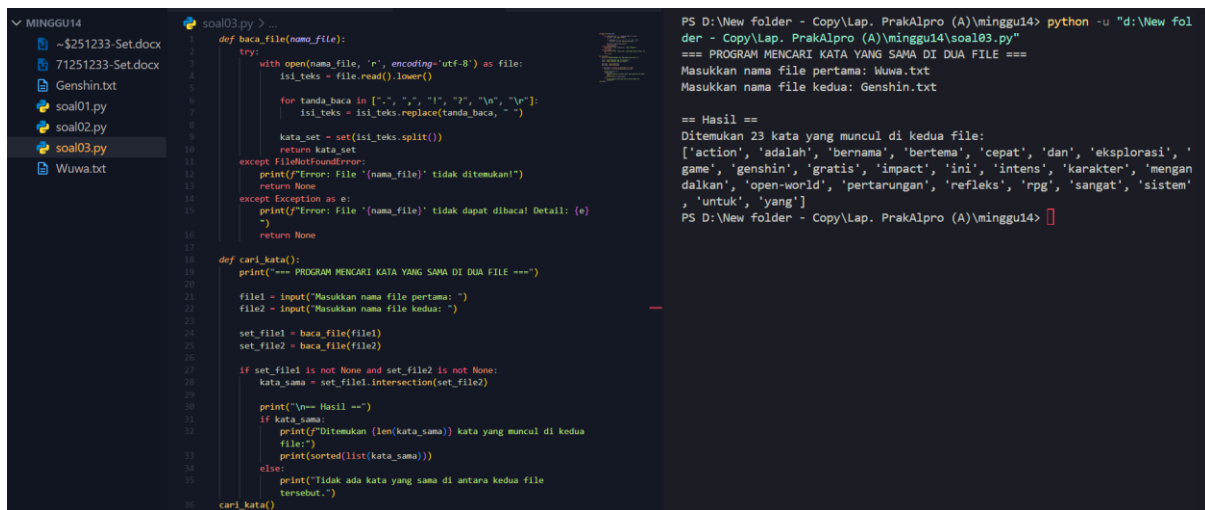
PS D:\New folder - Copy\Lap. PrakaIpro (A)\minggui4>
```

Gambar 2.4: Source Code soal 02

Penjelasan:

Ini adalah program yang mendemonstrasikan konversi dari beberapa tipe data. Program terdiri dari satu fungsi **konversi()** yang menunjukkan proses perubahan tipe data antara List, Set, dan Tuple. Pada bagian pertama, List ["apel", "jeruk", "mangga", "apel"] dikonversi menjadi Set menggunakan **set()**, sehingga elemen duplikat seperti "apel" otomatis dihapus dan menghasilkan data unik seperti {'mangga', 'apel', 'jeruk'}. Pada bagian kedua, Set {"kucing", "anjing", "kelinci"} diubah menjadi List menggunakan **list()**, namun urutan data tidak selalu sama karena Set tidak memiliki urutan tetap. Bagian ketiga mengubah Tuple (10, 20, 30, 20, 10) menjadi Set sehingga nilai yang berulang seperti 10 dan 20 dihapus dan tersisa {10, 20, 30}. Terakhir, Set {False, True, "Python"} dikonversi menjadi Tuple menggunakan **tuple()**, menghasilkan kumpulan data yang bersifat immutable atau tidak dapat diubah setelah dibuat.

SOAL 03



```
MINGGU14
~$251233-Set.docx
71251233-Set.docx
Genshin.txt
soal01.py
soal02.py
soal03.py
Wuwa.txt

soal03.py
1 def baca_file(nama_file):
2     try:
3         with open(nama_file, 'r', encoding='utf-8') as file:
4             isi_teks = file.read().lower()
5             for tanda_baca in [",", ".", "!", "?", "\n", "\r"]:
6                 isi_teks = isi_teks.replace(tanda_baca, " ")
7             kata_set = set(isi_teks.split())
8             return kata_set
9     except FileNotFoundError:
10        print(f"Error: File '{nama_file}' tidak ditemukan!")
11        return None
12    except Exception as e:
13        print(f"Error: File '{nama_file}' tidak dapat dibaca! Detail: {e}")
14        return None
15
16 def cari_kata():
17     print("=== PROGRAM MENCARI KATA YANG SAMA DI DUA FILE ===")
18     file1 = input("Masukkan nama file pertama: ")
19     file2 = input("Masukkan nama file kedua: ")
20
21     set_file1 = baca_file(file1)
22     set_file2 = baca_file(file2)
23
24     if set_file1 is not None and set_file2 is not None:
25         kata_sama = set_file1.intersection(set_file2)
26
27         print("\n== Hasil ==")
28         if kata_sama:
29             print(f"Ditemukan {len(kata_sama)} kata yang muncul di kedua file:")
30             print(sorted(list(kata_sama)))
31         else:
32             print("Tidak ada kata yang sama di antara kedua file tersebut.")
33     cari_kata()

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu14> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu14\soal03.py"
=== PROGRAM MENCARI KATA YANG SAMA DI DUA FILE ===
Masukkan nama file pertama: Wuwa.txt
Masukkan nama file kedua: Genshin.txt

== Hasil ==
Ditemukan 23 kata yang muncul di kedua file:
['action', 'adalah', 'bernama', 'bertema', 'cepat', 'dan', 'eksplorasi', 'game', 'genshin', 'gratis', 'impact', 'ini', 'intens', 'karakter', 'mengan dalkan', 'open-world', 'pertarungan', 'refleks', 'rpg', 'sangat', 'sistem', 'untuk', 'yang']
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu14>
```

Gambar 2.5: Source Code soal 03

Penejelasan:

Ini adalah program yang dapat membaca dua file teks dan menampilkan semua kata-kata yang muncul pada kedua file tersebut. Program ini digunakan untuk mencari kata-kata yang muncul di dua file teks dengan memanfaatkan operasi Set Intersection. Program memiliki dua fungsi utama, yaitu **baca_file(nama_file)** dan **cari_kata()**. Fungsi **baca_file()** bertugas membuka serta membaca isi file lalu mengubah seluruh teks menjadi huruf kecil menggunakan **.lower()** agar kata seperti "Game" dan "game" dianggap sama. Setelah itu, tanda baca seperti titik, koma, tanda seru, tanda tanya, dan baris baru dihapus dengan menggantinya menjadi spasi supaya kata tidak menempel dengan tanda baca. Teks kemudian dipecah menjadi kata-kata menggunakan **.split()** lalu diubah menjadi Set agar kata yang duplikat otomatis dihapus. Fungsi ini juga memiliki error handling untuk menangani kondisi file tidak ditemukan atau file gagal dibaca. Selanjutnya, fungsi **cari_kata()** meminta pengguna memasukkan dua nama file, membaca kedua file menjadi Set, lalu menggunakan **intersection()** untuk mencari kata yang ada di kedua file sekaligus. Contohnya, jika file pertama berisi kata {'action', 'game', 'rpg'} dan file kedua berisi {'game', 'rpg', 'gratis'}, maka hasil intersection adalah {'game', 'rpg'}. Terakhir, hasil kata yang sama diurutkan menggunakan **sorted()** dan ditampilkan ke terminal.